Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

Environment Setup

C:\Program Files\MongoDB\Server\\bin\mongod.cfg on Windows).

replication:
replSetName: myReplicaSet

net stop MongoDB && net start MongoDB # Windows

mongosh
rs.initiate()
rs.status()

```
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    newTermStartDate: ISODate('2026-05-09T14:21:38.859Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-05-09T14:21:38.934Z')
  },
  members: [
    {
      _id: 0,
      name: '127.0.0.1:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 34,
      optime: { ts: Timestamp({ t: 1778336498, i: 17 }), t: Long('1') },
      optimeDate: ISODate('2026-05-09T14:21:38.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1778336498, i: 17 }), t: Long('1') },
      optimeWrittenDate: ISODate('2026-05-09T14:21:38.000Z'),
      lastAppliedWallTime: ISODate('2026-05-09T14:21:38.927Z'),
      lastDurableWallTime: ISODate('2026-05-09T14:21:38.927Z'),
      lastWrittenWallTime: ISODate('2026-05-09T14:21:38.927Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: 'Could not find member to sync from',
      electionTime: Timestamp({ t: 1778336498, i: 2 }),
      electionDate: ISODate('2026-05-09T14:21:38.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778336498, i: 17 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778336498, i: 17 })
}
myReplicaSet [direct: primary] test>
```

```
Step-by-Step Instructions
Step 1: Define the Account Model
 Create the models/Account.js file:
```

```
    This file defines the Mongoose schema for an account, which includes an
accountNumber and a balance.


javascript
// models/Account.js
const mongoose = require('mongoose');
const accountSchema = new mongoose.Schema({
accountNumber: {
type: String,
required: true,
unique: true
},
balance: {
type: Number,
required: true,
default: 0
}
});

module.exports = mongoose.model('Account', accountSchema);


 Explanation:


       We import the mongoose library.
       We define an accountSchema with accountNumber (string, required, and
unique) and balance (number, required, and default value of 0).
    *   We export the Mongoose model named Account using the defined schema.
Step 2: Implement Transaction Routes
 Create the routes/transactionRoutes.js file:


    This file contains the Express routes for handling fund transfers using
MongoDB transactions.


javascript
// routes/transactionRoutes.js
const express = require('express');
const mongoose = require('mongoose');
const Account = require('../models/Account');
const router = express.Router();

router.post('/transfer', async (req, res) => {
const session = await mongoose.startSession();
```

```
session.startTransaction();

try {
const { fromAccountNumber, toAccountNumber, amount } = req.body;

// Find accounts
const fromAccount = await Account.findOne({ accountNumber: fromAccountNumber
}).session(session);
const toAccount = await Account.findOne({ accountNumber: toAccountNumber
}).session(session);

if (!fromAccount || !toAccount) {
await session.abortTransaction();
session.endSession();
return res.status(400).json({ message: 'Invalid account numbers' });
}

if (fromAccount.balance < amount) {
await session.abortTransaction();
session.endSession();
return res.status(400).json({ message: 'Insufficient funds' });
}

// Perform the transfer
fromAccount.balance -= amount;
toAccount.balance += amount;

await fromAccount.save({ session });
await toAccount.save({ session });

await session.commitTransaction();
session.endSession();

res.json({ message: 'Transfer successful' });
} catch (error) {
await session.abortTransaction();
session.endSession();
console.error('Transaction failed:', error);
res.status(500).json({ message: 'Transaction failed', error: error.message });
}
});

module.exports = router;
```

Explanation:

We import express, mongoose, and the Account model.

We create an Express router.
We define a POST route /transfer that handles the fund transfer.
We start a Mongoose session and begin a transaction using mongoose.startSession() and session.startTransaction().
We find the fromAccount and toAccount based on their account numbers, using .session(session) to associate the queries with the transaction.
We perform validation to ensure both accounts exist and that the fromAccount has sufficient funds. If validation fails, we abort the transaction using session.abortTransaction(), end the session with session.endSession(), and return an error response.
We update the balances of the accounts and save the changes to the database, associating the save operation with the transaction session using the { session } option.
If all operations succeed, we commit the transaction using session.commitTransaction() and end the session with session.endSession().
* If any error occurs during the transaction, we catch the error, abort the transaction using session.abortTransaction(), end the session with session.endSession(), and return an error response.

Step 3: Set up the Main Application

Create the index.js file:

This file sets up the Express application, connects to MongoDB, and mounts the transaction routes.

javascript

```
// index.js
const express = require('express');
const mongoose = require('mongoose');
const transactionRoutes = require('./routes/transactionRoutes');
const app = express();
const port = 3000;

app.use(express.json());

// MongoDB connection string
const dbURI = 'mongodb://localhost:27017/bankingApp'; // Replace with your MongoDB URI

mongoose.connect(dbURI, {
useNewUrlParser: true,
useUnifiedTopology: true,
// Comment out or remove useCreateIndex and useFindAndModify
// if not needed for your MongoDB version
})
.then(() => {
console.log('Connected to MongoDB');
app.listen(port, () => {
```

```
console.log(Server is running on port ${port});
});
})
.catch((err) => {
console.error('MongoDB connection error:', err);
});

app.use('/api/transactions', transactionRoutes);


 Explanation:


       We import express, mongoose, and the transactionRoutes.
       We create an Express application and set the port.
       We use express.json() middleware to parse JSON request bodies.
       We define the MongoDB connection URI.  Important: Replace
'mongodb://localhost:27017/bankingApp' with the correct connection string for
your MongoDB instance. Ensure the database name is bankingApp or change it
accordingly.
       We connect to MongoDB using mongoose.connect().
       We start the Express server after successfully connecting to MongoDB.
    *   We mount the transactionRoutes at the /api/transactions endpoint.
Step 4: Populate Initial Data
 Insert initial account data into the database:


    You can use the mongo shell to insert initial data.  First, connect to
your MongoDB instance:


bash
mongo

    Then, switch to the bankingApp database:


javascript
use bankingApp

    Insert two initial accounts:


javascript
db.accounts.insertMany([
{ accountNumber: "12345", balance: 1000 },
{ accountNumber: "67890", balance: 500 }
])
```

Explanation:

This step creates two accounts with initial balances. These accounts will be used for testing the transaction functionality.

Step 5: Testing the Transaction

Start the Node.js application:

```
bash
node index.js
```

Send a POST request to the /api/transactions/transfer endpoint using a tool like curl or Postman:

```
bash
curl -X POST -H "Content-Type: application/json" -d '{
"fromAccountNumber": "12345",
"toAccountNumber": "67890",
"amount": 200
}' http://localhost:3000/api/transactions/transfer
```

Expected Output:

A successful transfer should return a JSON response:

```
json
{
"message": "Transfer successful"
}
```

Verify the account balances in the database:

Use the mongo shell to query the accounts:

```
javascript
db.accounts.find()
```

The balances should be updated as follows:

```
json
```

```
[
{
"_id": ObjectId("..."),
"accountNumber": "12345",
"balance": 800,
"__v": 0
},
{
"_id": ObjectId("..."),
"accountNumber": "67890",
"balance": 700,
"__v": 0
}
]
```

Test a failed transaction:

Send a transfer request with an amount greater than the available balance in the fromAccount:

```
bash
curl -X POST -H "Content-Type: application/json" -d '{
"fromAccountNumber": "12345",
"toAccountNumber": "67890",
"amount": 1000
}' http://localhost:3000/api/transactions/transfer
```

Expected Output:

The server should return an error response:

```
json
{
"message": "Insufficient funds"
}
```

Verify the account balances in the database (after the failed transaction):

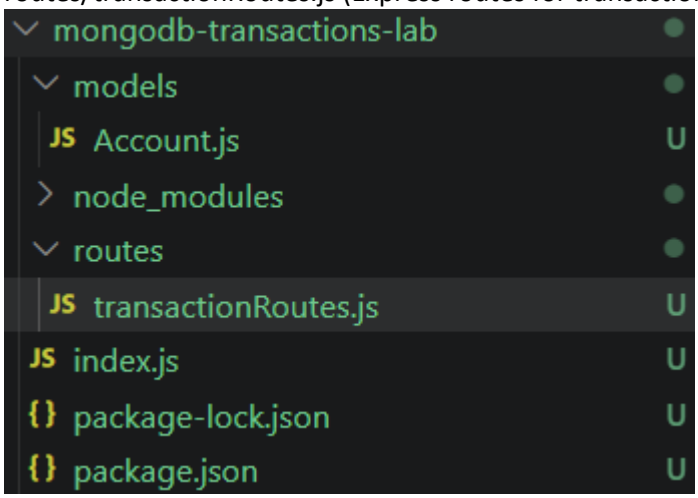
Use the mongo shell to query the accounts:

```
javascript
db.accounts.find()
```

```
    The balances should remain unchanged:


json
[
{
"_id": ObjectId("..."),
"accountNumber": "12345",
"balance": 800,
"__v": 0
},
{
"_id": ObjectId("..."),
"accountNumber": "67890",
"balance": 700,
"__v": 0
}
]
```

mkdir mongodb-transactions-lab
cd mongodb-transactions-lab
npm init -y
npm install express mongoose mongodb
index.js (main application file)
models/Account.js (Mongoose model for accounts)
routes/transactionRoutes.js (Express routes for transactions)



Answer:

```
myReplicaSet [direct: primary] test> use bankingApp
switched to db bankingApp
myReplicaSet [direct: primary] bankingApp> db.accounts.insertMany([
| { accountNumber: "12345", balance: 1000 },
| { accountNumber: "67890", balance: 500 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69ff44b0bf3893fb293682d1'),
    '1': ObjectId('69ff44b0bf3893fb293682d2')
  }
}
myReplicaSet [direct: primary] bankingApp>
```

POST http://localhost:3000/api/transactions/transfer Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
"fromAccountNumber": "12345",
"toAccountNumber": "67890",
"amount": 200
}
```

Status: 200 OK Size: 45 Bytes Time: 39 ms

Response Headers 6 Cookies Results Docs </>

```
{
  "message": "Transfer completed successfully"
}
```

```
myReplicaSet [direct: primary] bankingApp> db.accounts.find()
[
  {
    _id: ObjectId('69ff44b0bf3893fb293682d1'),
    accountNumber: '12345',
    balance: 800
  },
  {
    _id: ObjectId('69ff44b0bf3893fb293682d2'),
    accountNumber: '67890',
    balance: 700
  }
]
myReplicaSet [direct: primary] bankingApp>
```

POST http://localhost:3000/api/transactions/transfer Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
"fromAccountNumber": "12345",
"toAccountNumber": "67890",
"amount": 1000
}
```

Status: 400 Bad Request Size: 34 Bytes Time: 9 ms

Response Headers 6 Cookies Results Docs </>

```
{
  "message": "Insufficient balance"
}
```

If failed no changes happen in the database

```
myReplicaSet [direct: primary] bankingApp> db.accounts.find()
[
  {
    _id: ObjectId('69ff44b0bf3893fb293682d1'),
    accountNumber: '12345',
    balance: 800
  },
  {
    _id: ObjectId('69ff44b0bf3893fb293682d2'),
    accountNumber: '67890',
    balance: 700
  }
]
myReplicaSet [direct: primary] bankingApp>
```